Production Security & Architecture Audit

Read-only inspection of the GO7 codebase, Firestore rules, Cloud Functions, payments, and operations. No production code was modified during this audit.

Project: go7-app-209e4 | Live: go7.ng | Audit date: 2026-08-08
Evidence standard: path / rule / function — not generic checklists

38	28	52
Production readiness	Security	Reliability
34	**30**	**18**
Payment security	Data protection	Testing

Executive verdict

> **GO7 is live but not security-production-ready.** The product operates at go7.ng with real payments, but critical access-control and payment-trust failures mean a motivated client can forge paid status, mutate fees/earnings, and access cross-tenant data. Treat P0 as urgent production hardening.

1. Rotate Paystack `sk_live` + Infobip; scrub repo/history
2. Auth-gate or remove open money CFs (`verifyPayment` , `backfillPayment` , `updateOrderOnPaymentVerification` , agent earnings)
3. HMAC on `onPaystackTransferUpdate`
4. Stop client `paymentStatus: paid` — server verify + amount bind
5. `debitWalletForDelivery` : ignore client fee; server fare
6. Remove Firestore catch-all; lock bookings/orders fields + transitions
7. CF-only vendor/rider balances; field denylist
8. Deploy participant chat rules; scope rider updates to assigned
9. `users` get own/admin only; lock addresses; Storage proof party-only
10. Scheduled Firestore export + restore runbook; enable Sentry

1. Architecture (summary)

Area	Status	Evidence
Frontend	IMPLEMENTED	React 18 + Vite 7 SPA; Hostinger FTP deploy
Cloud Functions	IMPLEMENTED	functions/ Node 22: wallet, Paystack, OTP, bills
Firestore	IMPLEMENTED	Live rules match repo firestore.rules
Client vs server trust	UNSAFE	Orders/bookings/status/fees largely client-written
Deploy	IMPLEMENTED	npm run deploy:hostinger zero-downtime FTP

2–4. Auth / RBAC / conceptual access

Test	Prevented?	Evidence
Customer reads another order	NO	orders Go7-* public read; any-auth + catch-all
Customer sets paymentStatus paid	NO	Popup callback writes paid without server verify
Any rider updates any order	NO	isRider() with no assignment check
Vendor product cross-write	YES (path)	vendors/{id}/products requires isOwner
Vendor inflate balance / payout	NO	Unrestricted vendor write + client withdrawal
Client sets role=admin	YES	users create/update block admin self-promote
Duplicate wallet webhook	YES	wallet.js reference idempotency + HMAC
Duplicate verifyPayment HTTP	NO	Weak/no lock on legacy path

5. Firestore security (highlights)

> **Catch-all:** `match /{document=**} { allow read, write: if request.auth != null; }` (firestore.rules ~726–728)

Collection	Issue
bookings	World read/create; any auth update
orders	Public Go7-* read; rider/owner/vendor update no field lock
users	get: if true (world); list: authenticated
chats/messages	Any authenticated read/write
vendors	Owner write unrestricted (balances included)
riders	Self update all fields; read any auth
wallets	Balance fields protected for owner (better island)

6–9. API / payments / PCI

Control	Status	Evidence
Wallet Paystack webhook	OK	HMAC + re-verify + idempotent docs (wallet.js)
Transfer webhook	UNSAFE	onPaystackTransferUpdate — no HMAC
verifyPayment / backfillPayment	UNSAFE	Unauthenticated credit paths
Client payment amount/status/fee	UNSAFE	BookDelivery / FoodCheckout / debit fee trust
PCI raw card data	OK	Paystack Popup only — no CVV/PAN forms
Secrets in repo	UNSAFE	sk_live in test-dva-webhook.js; Infobip tracked
Currency NGN assert	MISSING	Client sends NGN; server does not reject mismatch

Public vs secret keys

Credential	Class	OK to expose?
Paystack pk_*	PUBLIC	Yes (domain-restrict)
Paystack sk_* / webhook HMAC	SECRET	NEVER
Firebase web apiKey	PUBLIC	Yes + App Check
Service account / Infobip / Termii / FTP password	SECRET	NEVER
MapTiler / Mapbox browser keys	PUBLIC	Yes if referrer-restricted
VITE_*SECRET*	SECRET (misplaced)	Never put real secrets in Vite

10–14. Fraud / location / rider / vendor / customer

Area	Status	Key finding
Fraud controls	MISSING	No App Check, velocity, fingerprint; wallet idempotency exists
BookDelivery location	OK	GPS + Abeokuta bounds; address↔coords
TrackDelivery	UNSAFE	Fake riders / ETA demos still present
Android rider nav	OK	Navigation SDK follow camera
Rider earnings/completion	UNSAFE	Client completeDelivery credits earnings
Vendor wallet	UNSAFE	Client mutates availableBalance
Customer isolation	UNSAFE	users get world; bookings public; chats any-auth

16. Order state machine

> **No server-enforced transition graph.** Status dialects are fragmented (food, GoBuy, rider, vendor, paystackConfig). Customer/vendor can mark delivered; unpaid→completed and cancelled→active are possible via rules.

17–22. Uploads / chat / notifications / audit / errors / secrets

Area	Status	Evidence
Hostinger uploads	OK	Auth, MIME, 5MB, path owner
Storage proof images	UNSAFE	Any authenticated write
Chat rules	UNSAFE	Any auth; stricter firestore.chat.rules not deployed
In-app notifications	OK	Owner-scoped; CF create only
WA/OTP send callables	UNSAFE	Under-authorized / open
Immutable audit log	MISSING	adminActions/audit_logs not locked in live rules
Error handling	PARTIAL	Many CFs return error.message

23–31. Ops (deps, CI, DR, monitoring, deploy)

Area	Status	Evidence
npm audit	UNSAFE	Root 1 critical / 15 high (snapshot 2026-08-08)
CI/CD	MISSING	No GitHub Actions; manual FTP + firebase deploy
Backups / DR	MISSING	Ad-hoc backups/ dump; no RPO/RTO/restore runbook
Monitoring	MISSING	Sentry init off/mocked; thin adminAlerts
Testing	MISSING	~10 Jest files; no rules/security/E2E suites
Deploy	OK	Zero-downtime Hostinger asset→verify→HTML cutover

Rollback	**PARTIAL**	Possible via prior dist / redeploy; undocumented emergency runbook

32. OWASP-style review

Category	Status	Risk	Recommendation
A01 Broken access control	UNSAFE	Critical	Deny-by-default; party-scoped chats; CF authz
A02 Cryptographic failures	PARTIAL	High	Rotate secrets; timing-safe HMAC; App Check
A03 Injection	PARTIAL	Medium	Escape Swal HTML; gate OTP senders
A04 Insecure design	UNSAFE	Critical	Server commerce ledger + state machine
A05 Misconfiguration	UNSAFE	High	Catch-all / CORS / weak WA token
A06 Vulnerable components	UNSAFE	High	Patch critical/high; audit CI
A07 Auth failures	PARTIAL	High	Agent Auth; gate open CFs
A08 Integrity failures	UNSAFE	Critical	HMAC all webhooks; CF-only balances
A09 Logging/monitoring	MISSING	High	Immutable audit + Sentry
A10 SSRF	PARTIAL	Medium	Auth + rate-limit Distance Matrix

33. Final scorecard (condensed)

Category	Status	Risk	Required action
Architecture	PARTIAL	High	Server-authoritative commerce
Authentication	PARTIAL	High	Agent Firebase Auth
Authorization / Firestore / API	UNSAFE	Critical	RBAC + deny default + CF authz
Payments	UNSAFE	Critical	Verify + bind amount server-side
PCI	OK	Low	Remove VITE secret pattern
Order state machine	MISSING	Critical	Server transitions
Chat	UNSAFE	High	Deploy participant rules
Secrets	UNSAFE	Critical	Rotate + scrub
Backups / DR	MISSING	Critical	Scheduled export + runbook
Monitoring / Testing / CI	MISSING	High	Sentry + rules tests + CI
Deployment	OK	Low	Keep zero-downtime order
Rollback	PARTIAL	High	Document emergency rollback

34. P0 — critical findings

Problem	Location	Fix	Func impact?
Live Paystack + Infobip secrets	test-dva-webhook.js; functions/.env.infobip	Rotate; scrub; untrack	Config update only
Unauthenticated money CFs	index.js; agentEarnings.js	Auth+Admin or delete	Admin must use Auth
Unsigned transfer webhook	vendorPayouts.js:585+	HMAC like wallet.js	None if signed already

Client paid + fee trust	BookDelivery/FoodCheckout; wallet.js	Server verify; ignore fee	Checkout awaits verify
Catch-all + open bookings	firestore.rules 395–401, 726–728	Deny default; field-lock	Inventory client writes first
Vendor/rider financial self-write	rules + vendorWalletUtils + riderService	CF-only balances	Move credits to CF

36. Final Q&A

1. **Production-ready?** NO for security. YES as a running product with known critical risk.
2. **Key vulns?** Catch-all rules; open money CFs; client paid/fee; unsigned transfer webhook; leaked secrets; open chats; balance self-writes.
3. **Incomplete?** State machine; immutable audit; DR; App Check; CI; Sentry; rules tests; agent Auth.
4. **Unauthorized access?** Cross-customer users/bookings/chats; any-rider orders; vendor self-balance; open admin CFs.
5. **Client money/status?** YES.
6. **Firestore rules secure?** NO.
7. **Paystack webhooks?** Wallet YES (HMAC+idempotent). Transfer NO. Legacy verify NO.
8. **Secrets protected?** NO.
9. **Location reliable?** BookDelivery mostly yes; TrackDelivery demos undermine trust.
10. **Rider nav ready?** Android SDK yes; GPS spoof via rules; web tracking mixed.
11. **Logging / monitoring / DR / tests?** All insufficient.
12. **Must fix?** Entire P0 list.
13. **After?** CI, Sentry, DR drills, App Check, status unification, pagination, docs.

> **Audit-only:** No code, rules, payments, auth, dependencies, deploy, or environment variables were modified. Interactive evidence tables also live in the Cursor canvas: go7-production-audit.canvas.tsx